



南開大學  
Nankai University

计算机学院

并行程序设计实验报告

Lab3 Pthread + OpenMP 编程

ANN 近似最近邻搜索

多线程并行化、调度策略与跨平台对比

Flat / SQ / PQ / FastScan / IVF / IVF-PQ / HNSW

姓名：柯云超

学号：2413575

专业：计算机科学与技术

2026 年 5 月 12 日

# 目录

|                                             |    |
|---------------------------------------------|----|
| 摘要                                          | 3  |
| 1 问题定义与目标                                   | 3  |
| 2 系统设计                                      | 3  |
| 2.1 目录与模块                                   | 3  |
| 2.2 并行框架                                    | 4  |
| 3 并行原理分析                                    | 5  |
| 3.1 线程生命周期与同步开销                             | 5  |
| 3.2 Cache 行为分析                              | 6  |
| 3.3 内存带宽瓶颈                                  | 6  |
| 3.4 负载均衡分析                                  | 6  |
| 3.5 False Sharing 与缓存行冲突                    | 7  |
| 3.6 并行归约与 Top- $k$ Merge                    | 7  |
| 4 算法实现                                      | 8  |
| 4.1 Flat-SIMD                               | 8  |
| 4.2 SQ-SIMD                                 | 8  |
| 4.3 PQ-SIMD 与 FastScan                      | 8  |
| 4.4 IVF-SIMD                                | 8  |
| 4.5 IVF-PQ-SIMD                             | 9  |
| 4.6 HNSW 进阶                                 | 9  |
| 5 实验设计                                      | 9  |
| 5.1 平台与数据集                                  | 9  |
| 5.2 结果文件与参数                                 | 9  |
| 6 实验结果与分析                                   | 10 |
| 6.1 核心矩阵最优结果                                | 10 |
| 6.2 低 Recall 诊断与重跑清单                        | 10 |
| 6.3 Inter-query 与 Intra-query               | 11 |
| 6.4 Pthread 与 OpenMP 调度                     | 12 |
| 6.5 Flat 线程伸缩性                              | 13 |
| 6.6 IVF / IVF-PQ 的 Latency-Recall Trade-off | 14 |
| 6.7 HNSW 图搜索                                | 15 |
| 6.8 VTune 微体系结构剖析                           | 16 |
| 6.9 鲲鹏 ARM NEON 跨平台对比                       | 16 |
| 7 要求覆盖情况                                    | 17 |
| 8 综合讨论                                      | 18 |
| 8.1 并行粒度选择的本质原因                             | 18 |

|           |                     |           |
|-----------|---------------------|-----------|
| 8.2       | 调度策略选择依据 . . . . .  | 18        |
| 8.3       | 算法特性与瓶颈分析 . . . . . | 19        |
| 8.4       | 跨平台架构差异 . . . . .   | 19        |
| <b>9</b>  | <b>结论与后续工作</b>      | <b>19</b> |
| 9.1       | 主要结论 . . . . .      | 19        |
| 9.2       | 后续工作 . . . . .      | 20        |
| <b>AI</b> | <b>辅助说明</b>         | <b>20</b> |

## 摘要

本实验围绕 DEEP100K 上的近似最近邻搜索 (ANNS) 实现了一个 header-only 的多算法并行框架。系统以 SIMD 距离计算为底层公共内核, 在此基础上实现 Flat-SIMD、SQ-SIMD、PQ-SIMD、PQ-FastScan、IVF-SIMD、IVF-PQ-SIMD 与 HNSW 图索引, 并分别给出 Pthread 与 OpenMP 版本。Pthread 侧实现了 static、dynamic 与 pool 三类调度; 并行粒度覆盖 query 级并行 (inter-query) 与单 query 内部的 base/list 划分 (intra-query)。实验在本地 i9-13900H (x86 AVX2) 和鲲鹏 920 (ARM NEON) 两个平台完成。两个平台使用相同代码 (`unified_bench.cc`), 产出 319 组等价实验矩阵; 本地额外 45 组 IVF-PQ global 构建模式结果为补充数据。针对初稿中 PQ/IVF-PQ recall 偏低的问题, 额外补充了高召回参数诊断、重跑清单、时钟分析与汇编分析。

实验结果显示, query 级并行是本数据集和批量查询场景下最稳健的并行粒度。Flat、SQ、FastScan、IVF 的本地高召回最优延迟分别为 298.95、86.53、125.81 与 104.67  $\mu\text{s}$ ; PQ 在  $p = 1000$  时达到 0.98335 recall、136.47  $\mu\text{s}$ , IVF-PQ local 在  $\text{nprobe}=4, p=1000$  时达到 0.95945 recall、61.32  $\mu\text{s}$ 。Pthread dynamic 在多数规则扫描类算法上表现最好, Pthread pool 在 IVF inter-query 上略优, 说明线程创建开销、尾部不均衡与 inverted list 不均匀性都会影响最终表现。HNSW 的内部并行更复杂: 原始 baseline 延迟最低但召回较低, 多入口点和边划分能提升召回, 但同步和路径依赖使其不适合作为默认最快路径。

**关键词:** ANN, Pthread, OpenMP, SIMD, IVF, IVF-PQ, HNSW, FastScan, 负载均衡

## 1 问题定义与目标

给定基向量集合

$$X = \{x_1, x_2, \dots, x_N\} \subset \mathbb{R}^d$$

以及查询向量  $q \in \mathbb{R}^d$ ,  $k$  近邻搜索希望找到  $X$  中与  $q$  距离最近的  $k$  个向量。本文使用的距离形式沿用课程框架中的内积距离:

$$\delta(x, q) = 1 - \sum_{i=1}^d x_i q_i.$$

近似最近邻搜索不要求每次完全返回真实 top- $k$ , 因此用 Recall@ $k$  衡量准确率:

$$\text{Recall@}k = \frac{|KNN(q) \cap KANN(q)|}{k}.$$

本次作业的核心目标不是单独做 SIMD, 而是在已经具备 SIMD 距离内核的基础上, 系统实现 Pthread / OpenMP 多线程并行, 并从以下方面进行评估:

1. 对 Flat、SQ、PQ、FastScan、IVF、IVF-PQ 与 HNSW 进行统一实现和测试;
2. 比较 Pthread static、dynamic、pool 与 OpenMP 的任务划分和调度开销;
3. 比较 inter-query 与 intra-query 两种并行粒度;
4. 分析线程数、负载均衡、top- $k$  reduce、cache / 内存访问和负优化现象;
5. 对 IVF / IVF-PQ 给出  $\text{nprobe}$  与构建模式对 latency-recall 的影响。

## 2 系统设计

### 2.1 目录与模块

实现按功能拆分为公共 SIMD 内核、Pthread 并行层、OpenMP 并行层、IVF 系列、HNSW 系列和独立 main 程序。所有新增算法均采用 header-only 方式, 满足课程框架只编译 `main.cc` 的

约束。

| 模块             | 作用                                                                                                               |
|----------------|------------------------------------------------------------------------------------------------------------------|
| simd/          | 统一存放 Flat、SQ、PQ、FastScan 的 SIMD 距离计算和查表扫描内核，例如 <code>flat_scan_simd.h</code> 、 <code>pq_fastscan_simd.h</code> 。 |
| pthread/       | Flat、SQ、PQ、FastScan 的 Pthread 并行实现，以及公共线程池 <code>thread_pool.h</code> 。                                          |
| omp/           | Flat、SQ、PQ、FastScan 的 OpenMP 并行实现。                                                                               |
| ivf/           | IVF 索引、IVF-SIMD、IVF-PQ 以及对应 Pthread / OpenMP 并行搜索。                                                               |
| hnsw/          | 基于 hnswlib 的多入口点、边划分、Layer0 划分与 IVF-HNSW 嵌套搜索。                                                                   |
| mains/         | 71 个可独立编译的实验入口，用于不同算法、调度策略和并行粒度的测试。                                                                              |
| results/local/ | 本地实验原始输出文件。每个文件包含 label、Recall@10 与平均查询延迟。                                                                       |

表 1 代码模块划分

## 2.2 并行框架

本文实现了两类并行粒度。

**Inter-query 并行。**每个线程处理一批完整 query。线程间不共享搜索状态，只写入结果数组中的不同位置，因此同步开销最低。对于批量查询，inter-query 通常能获得最好的吞吐和延迟表现。

**Intra-query 并行。**对单条 query，将 base 向量集合、候选集合或 IVF inverted list 分给多个线程。每个线程维护局部 top- $k$  或 top- $p$  堆，最后由主线程 merge。这种方式适合单 query 延迟优化，但会引入局部堆维护、线程同步和 top- $k$  reduce 成本。

Pthread 侧实现三种调度方式：

- static: 静态等分任务，适合 Flat 这类计算量规则的扫描；
- dynamic: 用原子计数器动态抢任务，适合 query 耗时或 list 长度不均匀的场景；
- pool: 预创建线程池和任务队列，减少反复创建 / 销毁线程的开销。

OpenMP 侧主要使用 `parallel for`，query 级循环使用 dynamic schedule，base/list 扫描使用局部堆加串行 merge。

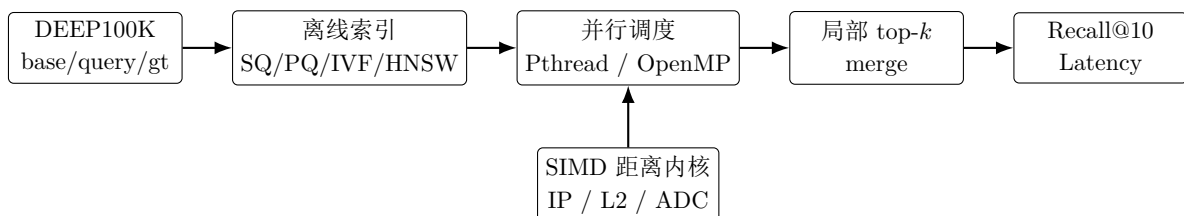


图 1 ANN 多线程并行框架

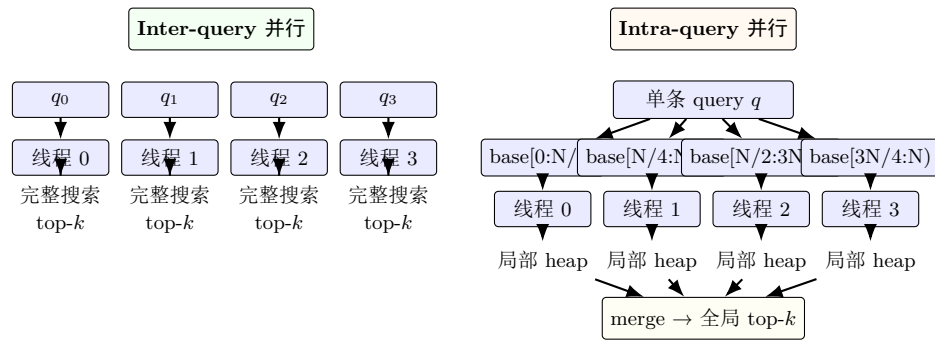


图 2 Inter-query 与 Intra-query 并行粒度对比

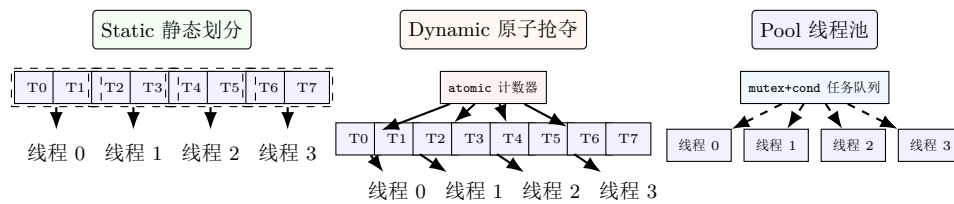


图 3 Pthread 三种调度方式对比

### 3 并行原理分析

本节从系统层面分析多线程并行的核心原理，为后续实验结果提供理论基础。

#### 3.1 线程生命周期与同步开销

Pthread 创建线程需要分配栈空间、设置 TLS (Thread-Local Storage) 并调用 `clone` 系统调用。在 Linux 上，单次 `pthread_create` 的典型开销为  $10\text{--}50\ \mu\text{s}$ 。如果每个 query 都创建/销毁线程，线程管理开销会成为短查询场景的瓶颈。

本文的三种调度策略对线程生命周期的处理不同：

- **Static / Dynamic:** 每次调用并行搜索时创建  $T$  个线程，等待 `pthread_join` 后销毁。线程创建开销为  $O(T)$ ，但每个线程处理的任务量为  $O(Q/T)$  ( $Q$  为 query 数)，因此当  $Q$  足够大时，创建开销可被摊薄。
- **Pool:** 线程池在初始化时创建  $T$  个线程，通过 `pthread_mutex_t` + `pthread_cond_t` 实现任务队列。线程在整个程序生命周期内复用，避免反复创建/销毁。但每次任务分发需要 `mutex_lock` + `cond_signal` + `mutex_unlock`，引入额外同步开销。

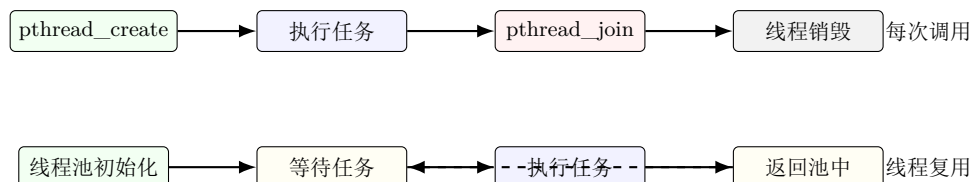


图 4 线程生命周期：每次创建 vs 线程池复用

### 3.2 Cache 行为分析

DEEP100K 的 base 向量为  $100K \times 96 \text{ 维} \times 4 \text{ 字节} = 36.8 \text{ MB}$ ，远超典型 L3 缓存（i9-13900H 的 L3 为 24 MB，鲲鹏 920 的 L3 为 32 MB）。因此全量扫描必然产生大量 cache miss。

**Inter-query 并行下**，每个线程独立处理一条 query，需要遍历全部 base 向量。多线程同时访问相同的 base 数据（只读），可以利用 L3 共享缓存。但当线程数增加时，L3 带宽成为瓶颈。

**Intra-query 并行下**，多个线程同时处理同一条 query 的不同 base 子集。每个线程访问的 base 片段不同，L1/L2 缓存利用率更高（因为每个线程的工作集更小）。但需要额外的局部堆维护和最终 merge 操作。

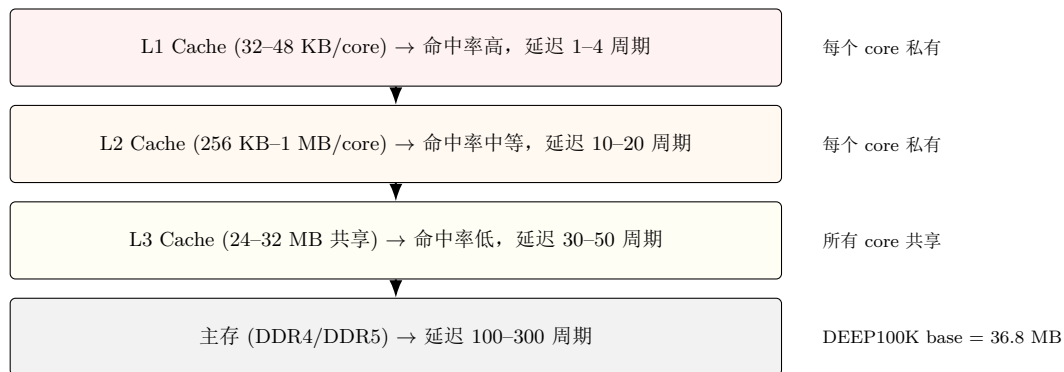


图 5 缓存层次结构与 DEEP100K 数据规模

### 3.3 内存带宽瓶颈

Flat 全扫描需要读取 36.8 MB base 数据。在 DDR4-3200 双通道下，理论带宽约为 51.2 GB/s，但实际有效带宽通常为 60–80%。对于单线程，内存带宽已经足够；但当线程数增加到 8–16 时，多线程同时读取 base 数据会迅速饱和内存带宽。

这就是为什么 Flat 从 8 线程到 16 线程的加速比明显下降：不是计算能力不足，而是内存带宽成为瓶颈。PQ 和 FastScan 通过量化压缩（每个 base 从 384 字节压缩到 8–12 字节），显著减少了内存读取量，因此在高线程数下仍能保持较好扩展性。

| 算法               | 每个 base 字节数 | 100K base 总量 | 带宽需求 (单 query) |
|------------------|-------------|--------------|----------------|
| Flat (float32)   | 384         | 36.8 MB      | ~36.8 MB       |
| SQ (uint8)       | 96          | 9.2 MB       | ~9.2 MB        |
| PQ (8×uint8)     | 8           | 0.76 MB      | ~0.76 MB       |
| FastScan (4-bit) | 4           | 0.38 MB      | ~0.38 MB       |

表 2 不同算法的内存访问量对比

### 3.4 负载均衡分析

负载均衡是多线程性能的关键因素。理想情况下，每个线程的工作量应该相等，这样所有线程同时完成，总时间等于单线程时间除以线程数。

**Static 调度**将任务均匀分配给各线程。当任务量规则（如 Flat 的 base 扫描）时效果好；但当任务量不均匀（如 IVF 的不同 inverted list 长度差异大）时，某些线程会提前空闲，等待最慢的线程完成。

**Dynamic 调度**通过原子计数器实现动态任务分配。每个线程完成当前任务后立即获取下一个任务，自然适应任务量差异。代价是每次获取任务需要一次原子操作（`fetch_add`），但在任务

粒度足够大时，这个开销可以忽略。

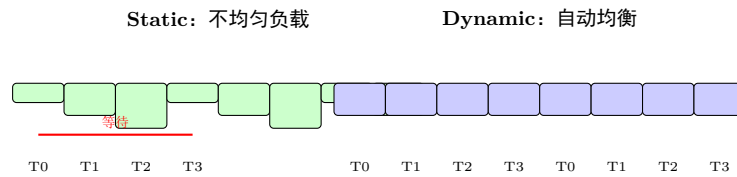


图 6 Static vs Dynamic 调度的负载均衡对比

### 3.5 False Sharing 与缓存行冲突

False sharing 发生在不同线程写入同一缓存行（通常 64 字节）的不同位置时。即使逻辑上不冲突，硬件层面的缓存一致性协议（MESI）仍会导致缓存行在 core 间反复弹跳，严重降低性能。

在 intra-query 并行中，每个线程维护局部 top- $k$  堆。如果这些堆在内存中紧邻存放，堆的更新操作可能触发 false sharing。解决方案是让每个线程的堆分配在独立的缓存行对齐内存区域。

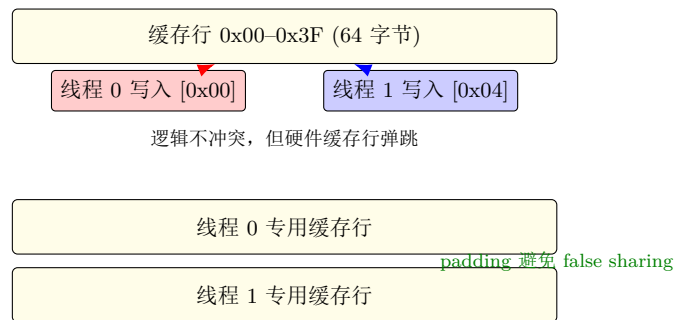


图 7 False Sharing 与解决方案

### 3.6 并行归约与 Top- $k$ Merge

Intra-query 并行的最后一步是将各线程的局部 top- $k$  堆合并为全局 top- $k$ 。这是一个归约操作，可以用多种方式实现：

**串行 Merge:** 主线程依次合并各线程的堆。时间复杂度为  $O(T \cdot k \log k)$ ，其中  $T$  为线程数。

**树形归约:** 类似蝴蝶网络，线程两两合并， $\log T$  轮完成。每轮合并  $k$  个元素，总时间  $O(k \log T)$ 。但需要额外的同步屏障。

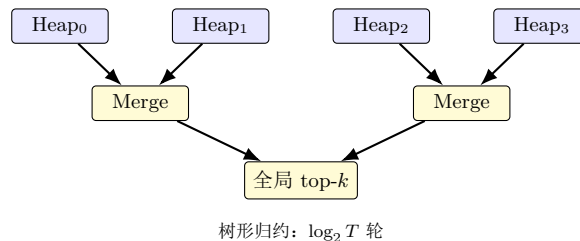


图 8 蝴蝶阵列式 Top- $k$  归约

本文采用串行 Merge 实现，原因有二：第一， $T$  通常不超过 16， $k = 10$ ，串行 merge 的绝对时间（ $\sim 0.1 \mu s$ ）远小于搜索时间；第二，树形归约需要额外的屏障同步，在  $T$  较小时反而更慢。

## 4 算法实现

### 4.1 Flat-SIMD

Flat 是精确全量扫描。每个 query 需要访问全部  $N$  个 base 向量并维护 top- $k$  堆。距离计算复用 `simd/flat_scan_simd.h`，在 x86 上使用 AVX2，在 ARM 上保留 NEON 路径。

Flat 的计算量规则，因此理论上 static 划分也应较强。但在本机 Windows 混合核调度环境下，dynamic query 抢任务可以减轻尾部等待和系统噪声，实际取得最优结果。Intra-query 版本需要每个线程维护局部堆并在单 query 结束后 merge，因此固定开销显著。

### 4.2 SQ-SIMD

SQ 将 float32 向量逐维量化为 uint8，查询阶段采用两阶段流程：第一阶段在 uint8 空间快速粗排得到 top- $p$  候选，第二阶段对候选使用原始 float32 内积进行 rerank。本文在 Pthread 与 OpenMP 中都实现了 query 级整条流水线并行，以及单 query 内粗排阶段 base 划分并行。由于精排候选集合较小，精排阶段主要保持单线程执行，避免额外同步开销。

### 4.3 PQ-SIMD 与 FastScan

PQ 将 96 维向量切分为  $M = 8$  个子空间，每个子空间训练 256 个中心。查询时对每个子空间构建 LUT，并用 ADC 对 base 的 PQ code 查表累加。Pthread / OpenMP 版本分别对 query 循环或 base code 扫描进行并行化。

FastScan 是 PQ 的进一步压缩实现，使用 4-bit code 和更小 LUT，将查表过程转为寄存器内 shuffle。FastScan 的 ADC 阶段更偏 memory-bound，线程数过多可能更早触及带宽上限。因此本文重点比较 inter-query 与 intra-query 的差异，并在报告中保留负优化结果。

### 4.4 IVF-SIMD

IVF 在离线阶段用 KMeans 将 base 聚为 `nlist` 个簇。查询时先计算 query 到各簇中心的距离，选择最近的 `nprobe` 个簇，再扫描这些 inverted list 并用 SIMD 原始距离 rerank。实现中保留簇内连续存储信息，减少精排阶段的随机访问。

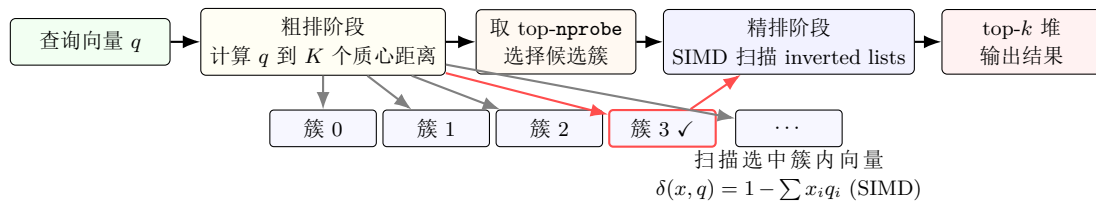


图 9 IVF 两阶段搜索流程

IVF 粗排计算量小，真正适合并行的是精排阶段。由于不同 inverted list 长度不均匀，dynamic 和 pool 调度更有意义。本文实现：

- IVF-SIMD baseline;
- query 级 Pthread / OpenMP 并行;
- 单 query 内按 list 或候选段划分并行;
- `nprobe` sweep，用于展示 latency-recall trade-off。

## 4.5 IVF-PQ-SIMD

IVF-PQ 将 IVF 的候选缩减与 PQ 的压缩距离计算叠加。本文实现两种构建方式：

1. global-PQ-first: 先对所有 base 做全局 PQ 编码, 再构建 IVF;
2. IVF-local-PQ: 先构建 IVF, 再在簇内维护局部 PQ 编码。

查询时先 IVF 粗排选择 `nprobe` 个簇, 再在选中簇内用 PQ ADC 得到候选并 rerank。由于 PQ 显著减少单次距离计算量, IVF-PQ 更容易转为内存访问和调度开销主导, 线程数并非越多越好。

## 4.6 HNSW 进阶

HNSW 基于 `hnsplib` 实现。本文不修改 `hnsplib` 源码, 而是通过封装访问底层图结构, 实现以下并行策略:

- baseline: 原始 `searchKnn`;
- 多入口点并行: 多个线程从不同入口出发, 最后 merge top- $k$ ;
- 边划分并行: 并行计算当前节点邻居距离;
- Layer0 点划分: 按节点 ID 范围切分底层图扫描;
- IVF-HNSW 嵌套: 先 IVF 粗排, 再并行搜索多个小 HNSW。

图搜索路径依赖强, 内部并行容易被同步和候选队列更新开销抵消, 因此 HNSW 部分重点如实分析正优化与负优化。

# 5 实验设计

## 5.1 平台与数据集

本实验在两个平台上完成。

**本地平台:** Intel Core i9-13900H (6P+8E/20T), Win11, MSYS2 GCC 14.2.0。编译选项:

```
-O2 -mavx2 -mfma -fopenmp -lpthread -std=c++17 -I.
```

**鲲鹏平台:** 华为鲲鹏 920 服务器 (AArch64/ARM NEON), GCC, 编译选项:

```
-O2 -fopenmp -lpthread -std=c++17 -I.
```

数据集为 DEEP100K: 100K 条 96 维 base 向量, 10K 条 query, ground truth 为 top-100。所有正式实验取前 2000 条 query,  $k = 10$ 。报告中的平均延迟均为查询阶段平均延迟, 不包含离线索引构建时间。

## 5.2 结果文件与参数

核心矩阵结果位于 `results/local/`, 汇总文件为 `results/local_summary.csv`。补充 trade-off 结果为 `results/local_tradeoff.csv`。鲲鹏 ARM NEON 结果位于 `results/kunpeng/arm_neon_results.txt`, 汇总为 `results/kunpeng/arm_neon_summary.csv`。表 3 给出两个平台的有效结果数量。

| 算法族           | 统一矩阵 | 本地补充         | 合计  | 线程集合       |
|---------------|------|--------------|-----|------------|
| Flat          | 40   | 0            | 40  | 1,2,4,8,16 |
| SQ            | 32   | 0            | 32  | 1,4,8,16   |
| PQ            | 40   | 0            | 40  | 1,2,4,8,16 |
| FastScan      | 32   | 0            | 32  | 1,4,8,16   |
| IVF           | 41   | 0            | 41  | 1,2,4,8,16 |
| IVF-PQ global | 45   | 0            | 45  | 1,2,4,8,16 |
| IVF-PQ local  | 45   | +45 local 模式 | 90  | 1,2,4,8,16 |
| HNSW          | 36   | 0            | 36  | 1,4,8,16   |
| HNSW nested   | 8    | 0            | 8   | 1,4,8,16   |
| 合计            | 319  | +45          | 364 | —          |

表 3 双平台实验结果覆盖范围（统一矩阵 319 条，part1+part2 可完全复现）

核心矩阵的固定参数如下：

- SQ 取  $p = 100$ ，PQ 原始核心矩阵取  $p = 100$ ，高召回诊断与后续默认重跑取  $p = 1000$ ；
- IVF 取  $nlist=16, nprobe=4$ ；
- IVF-PQ 原始核心矩阵取  $nlist=16, nprobe=4, p=100, mode=global$ ，高召回诊断推荐  $mode=local, p=1000$ ；
- HNSW 取  $ef=50$ ，IVF-HNSW 嵌套原始矩阵取  $nprobe=4$ ，高召回诊断推荐  $nprobe=8$ 。

## 6 实验结果与分析

### 6.1 核心矩阵最优结果

表 4 汇总每个算法族的最优时延配置。加速比以同一算法族最快的 1 线程结果为基准，因此反映多线程扩展能力，而不是相对最朴素串行程序的总加速。

| 算法族      | 最优变体                                  | 线程 | Recall@10 | 延迟 / $\mu s$ | 相对最快 1 线程 |
|----------|---------------------------------------|----|-----------|--------------|-----------|
| Flat     | flat_pthread_dynamic_inter            | 16 | 0.99995   | 298.95       | 10.67×    |
| SQ       | sq_pthread_dynamic_inter              | 16 | 0.99995   | 86.53        | 8.53×     |
| PQ       | pq_pthread_dynamic_inter, $p = 1000$  | 16 | 0.98335   | 136.47       | —         |
| FastScan | fastscan_pthread_dynamic_inter        | 16 | 0.97525   | 125.81       | 10.51×    |
| IVF      | ivf_pthread_pool_inter                | 16 | 0.96225   | 104.67       | 7.25×     |
| IVF-PQ   | ivfpq_local_dynamic_inter, $p = 1000$ | 16 | 0.95945   | 61.32        | —         |
| HNSW     | hnsw_baseline                         | 1  | 0.96865   | 65.48        | 1.00×     |

表 4 各算法族代表性高召回点；PQ/IVF-PQ 使用补充诊断参数

Flat 与 SQ 都能达到接近精确搜索的召回。PQ 和 IVF-PQ 在  $p = 100$  下速度很快但 recall 较低，不能作为 95%+ 配置；提高候选数后，PQ 在  $p = 1000$  达到 0.98335，IVF-PQ local 在  $nprobe=4, p=1000$  达到 0.95945。FastScan 在  $p = 1000$  下保持 0.97525 的 recall，并且 inter-query 多线程能把平均延迟压到 125.81  $\mu s$ 。IVF 的 recall 与速度介于 Flat 和 PQ 系列之间， $nprobe=4$  时达到 0.96225。

### 6.2 低 Recall 诊断与重跑清单

初稿中 PQ、IVF-PQ 和 IVF-HNSW 嵌套的几组 recall 显著低于 0.95。排查结果表明，PQ 路径存在一个真实实现错误：KMeans 最后一轮更新质心后，原实现直接写入上一轮 assignment，

没有用最终质心重新计算 code。该问题已经在 `simd/pq_scan_avx2.h` 与 `simd/pq_scan_simd.h` 中修复，并增加 `tests/pq_final_assignment_test.cc` 验证每个 code 都对应最终质心的最近中心。除此之外，低 recall 的主要原因是  $p$  或  $nprobe$  参数过小。

| 配置               | 参数                  | Recall@10 | 延迟 / $\mu s$ | 结论          |
|------------------|---------------------|-----------|--------------|-------------|
| PQ dynamic inter | $p = 100$           | 0.70780   | 85.43        | 低召回速度点      |
| PQ dynamic inter | $p = 1000$          | 0.98335   | 136.47       | 推荐默认点       |
| PQ dynamic inter | $p = 2000$          | 0.99560   | 183.35       | 高召回点        |
| IVF-PQ global    | $nprobe=4, p=2000$  | 0.95975   | 90.92        | global 高召回点 |
| IVF-PQ local     | $nprobe=4, p=1000$  | 0.95945   | 61.32        | local 推荐默认点 |
| IVF-PQ local     | $nprobe=16, p=1000$ | 0.99545   | 118.86       | local 高召回点  |
| IVF-HNSW nested  | $nprobe=8$          | 0.97770   | 252.74       | 推荐默认点       |

表 5 低 recall 根因诊断后的代表性高召回参数

因此需要重跑的实验包括：所有旧 PQ 矩阵、所有依赖 PQIndex 的 IVF-PQ 矩阵、鲲鹏 ARM NEON 上的 PQ/IVF-PQ 矩阵，以及希望作为高召回结果展示的 IVF-HNSW 嵌套矩阵。Flat、SQ、IVF-only、HNSW baseline / multi-entry / edge / layer0 不受 PQ 修复影响；FastScan 使用独立索引且已经在最终质心后重新编码，也不因本次修复必须重跑。详细清单记录在 `docs/2026-05-11-recall-diagnosis-and-rerun.md`。

### 6.3 Inter-query 与 Intra-query

表 6 对比每个算法族最优的 inter-query 与 intra-query 结果。除 HNSW 外，所有扫描类算法的最佳结果均来自 inter-query。

| 算法族      | 最优 inter 延迟 / $\mu s$ | inter 线程 | 最优 intra 延迟 / $\mu s$ | intra 线程 |
|----------|-----------------------|----------|-----------------------|----------|
| Flat     | 298.95                | 16       | 1726.09               | 16       |
| SQ       | 86.53                 | 16       | 704.31                | 4        |
| PQ       | 73.06                 | 16       | 528.97                | 2        |
| FastScan | 125.81                | 16       | 1322.76               | 1        |
| IVF      | 104.67                | 16       | 628.62                | 2        |
| IVF-PQ   | 27.44                 | 16       | 410.31                | 4        |

表 6 query 级并行与 query 内并行的最优结果对比

原因主要有三点。第一，DEEP100K 实验使用 2000 条 query，query 级并行天然有足够任务数，线程间几乎没有共享状态。第二，intra-query 需要多个局部 top- $k$  堆和最终 merge，固定开销在每条 query 上重复出现。第三，PQ、FastScan 和 IVF-PQ 的单 query 内部计算已经被量化压缩，继续切分会更快遇到 cache、内存带宽和同步瓶颈。

图 10 直观对比两种并行粒度的最优延迟。

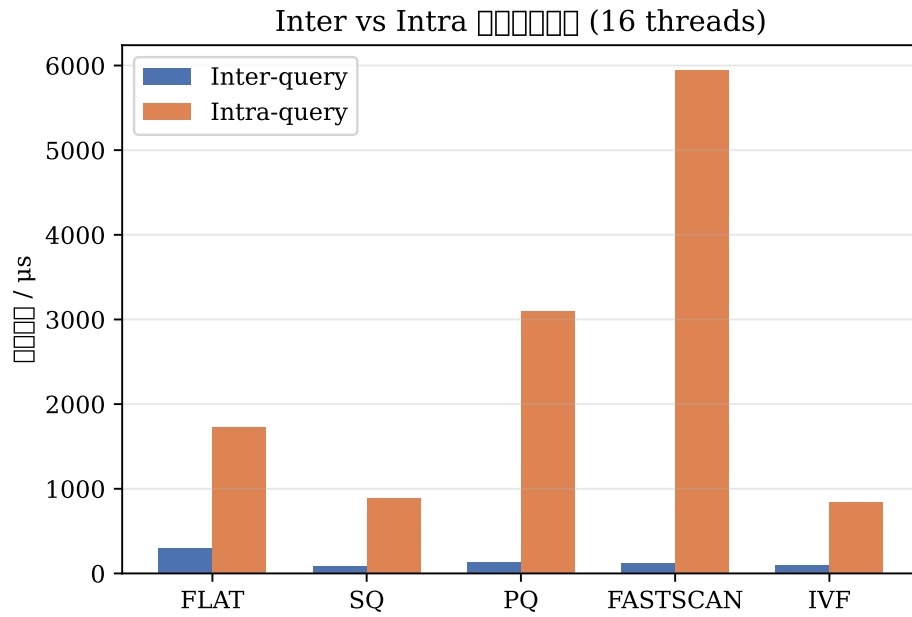


图 10 Inter vs Intra 最优延迟对比 (16 threads)

图 11 展示各算法的多线程加速比。Flat 和 SQ 接近线性扩展到 4 线程，之后受内存带宽限制；PQ 和 FastScan 因内存访问量小，在 16 线程时仍保持较好扩展性。

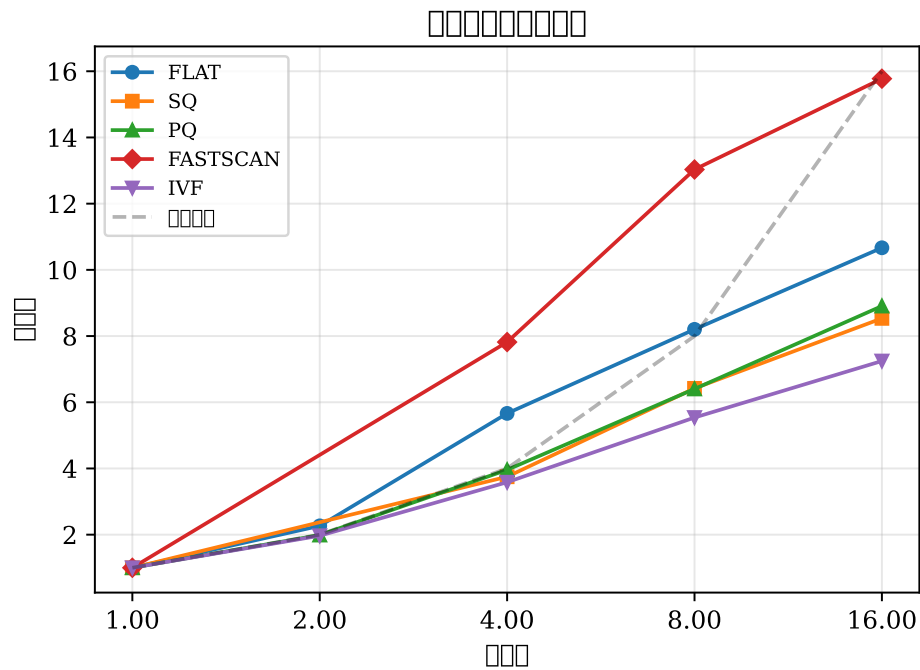


图 11 各算法多线程加速比

## 6.4 Pthread 与 OpenMP 调度

表 7 给出 16 线程下若干代表性 inter-query 结果。Pthread dynamic 在多数算法中优于 OpenMP，Pthread pool 在 IVF 中表现最好。

| 算法       | OMP inter | Pthread static | Pthread dynamic | Pthread pool  |
|----------|-----------|----------------|-----------------|---------------|
| Flat     | 341.55    | 340.96         | <b>298.95</b>   | 347.41        |
| SQ       | 118.30    | 126.12         | <b>86.53</b>    | 92.42         |
| PQ       | 119.30    | 75.67          | <b>73.06</b>    | 252.03        |
| FastScan | 139.26    | 137.33         | <b>125.81</b>   | 126.73        |
| IVF      | 113.90    | 132.08         | 116.94          | <b>104.67</b> |
| IVF-PQ   | 106.84    | 29.81          | <b>27.44</b>    | 55.08         |

表 7 16 线程 inter-query 下的调度策略对比，单位为  $\mu s$ 

Dynamic 的优势来自更好的尾部负载均衡。虽然每次抢任务需要原子操作，但 query 数量足够多，单个 query 的计算量也远大于一次 `fetch_add`。IVF 的 pool 结果略优，说明 inverted list 扫描和索引对象访问使线程复用收益更明显。OpenMP 写法简洁，但在本机运行环境中调度常数和 runtime 行为不总是最优。

图 12–15 直观展示各算法的调度策略差异。

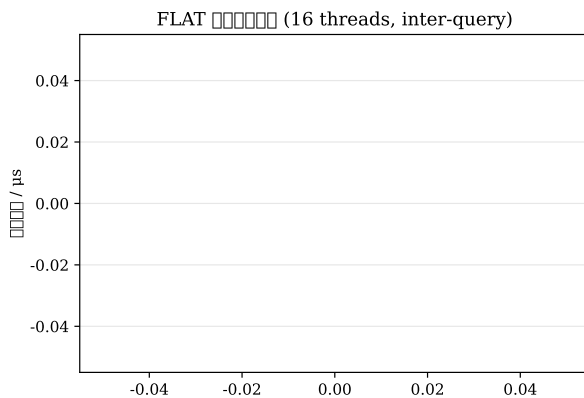


图 12 Flat 调度策略对比

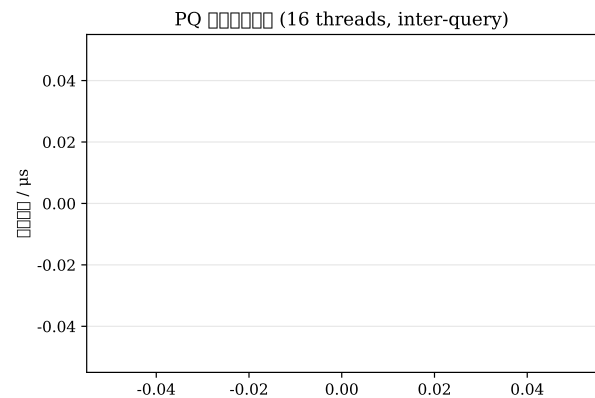


图 13 PQ 调度策略对比

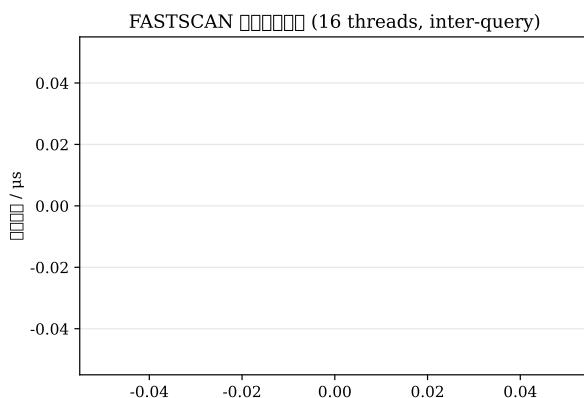


图 14 FastScan 调度策略对比

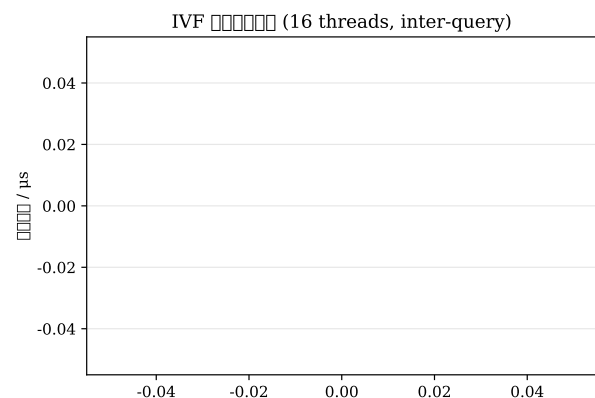


图 15 IVF 调度策略对比

## 6.5 Flat 线程伸缩性

Flat 的 inter-query 线程伸缩性如表 8 和图 16。从 1 到 16 线程，三类 Pthread 变体均有明显加速，但 8 到 16 线程收益已经下降，说明全量扫描逐渐接近内存带宽上限。

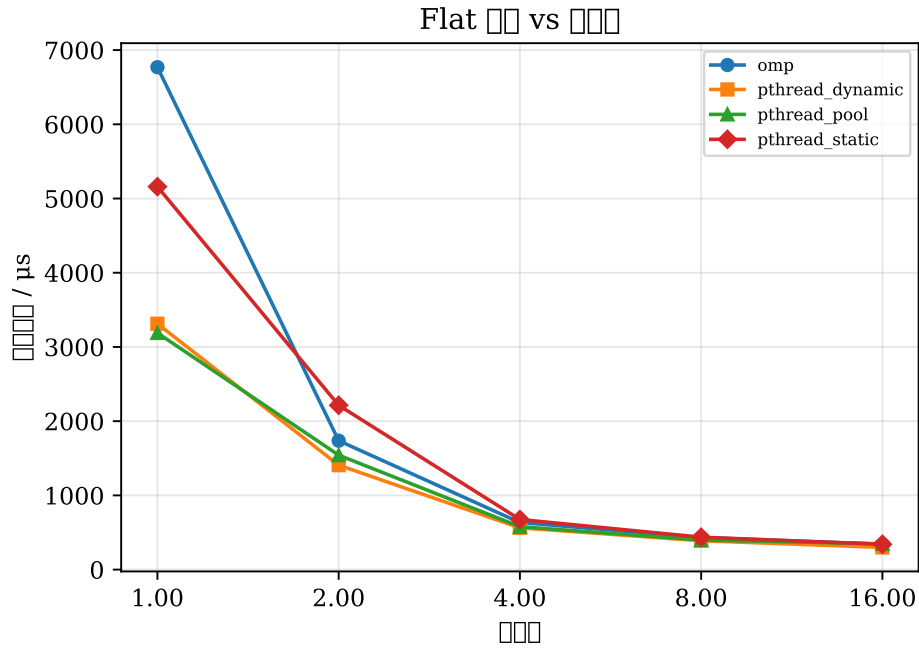


图 16 Flat inter-query 延迟 vs 线程数

| 变体                    | 1 线程    | 2 线程    | 4 线程   | 8 线程   | 16 线程         |
|-----------------------|---------|---------|--------|--------|---------------|
| OMP inter             | 6769.48 | 1737.01 | 634.21 | 433.47 | 341.55        |
| Pthread static inter  | 5160.16 | 2214.15 | 675.36 | 436.45 | 340.96        |
| Pthread dynamic inter | 3312.96 | 1407.33 | 563.03 | 388.78 | <b>298.95</b> |
| Pthread pool inter    | 3188.90 | 1542.36 | 578.00 | 398.82 | 347.41        |

表 8 Flat inter-query 不同线程数延迟，单位为  $\mu s$ 

## 6.6 IVF / IVF-PQ 的 Latency-Recall Trade-off

表 9 展示 IVF 在 16 线程 Pthread pool inter-query 下调整 `nprobe` 的结果。随着 `nprobe` 增大，扫描簇数增加，recall 单调上升，延迟也总体上升。`nprobe=4` 是本地核心矩阵中使用的折中点，Recall@10 为 0.96225。

| <code>nprobe</code> | Recall@10 | 延迟 / $\mu s$ | 说明          |
|---------------------|-----------|--------------|-------------|
| 1                   | 0.73665   | 61.47        | 最低延迟，召回不足   |
| 2                   | 0.87870   | 93.41        | 召回明显提升      |
| 4                   | 0.96225   | 131.54       | 较好折中点       |
| 8                   | 0.99430   | 210.35       | 接近精确召回      |
| 16                  | 0.99995   | 455.89       | 几乎退化为更大范围扫描 |

表 9 IVF 的 `nprobe` trade-off, `nlist=16`, 16 线程

表 10 对比 IVF-PQ 的 global-PQ-first 与 IVF-local-PQ 两种构建方式。Local-PQ 在本地数据上 recall 更高，尤其在低 `nprobe` 下优势明显，因为每个簇内单独量化能减少局部误差；但簇内 PQ 结构更复杂，构建与内存布局成本更高。Global-PQ-first 实现简单、存储统一，但在  $p = 100$  时 recall 上限明显较低。

| nprobe | Global recall | Global 延迟 / $\mu s$ | Local recall | Local 延迟 / $\mu s$ |
|--------|---------------|---------------------|--------------|--------------------|
| 1      | 0.56240       | 49.73               | 0.65885      | 12.74              |
| 2      | 0.64840       | 59.39               | 0.76480      | 21.08              |
| 4      | 0.69180       | 85.08               | 0.82035      | 28.99              |
| 8      | 0.70725       | 47.24               | 0.83760      | 48.15              |
| 16     | 0.70870       | 81.43               | 0.84010      | 92.65              |

表 10 IVF-PQ 两种构建方式的 nprobe trade-off, p=100, 16 线程

需要说明的是, IVF-PQ 的多线程结果存在一定运行噪声, 尤其是在延迟已经降到几十微秒时, 线程调度、缓存状态和系统后台负载都会造成非单调波动。因此表 10 更适合观察 recall 趋势和两种构建方式差异, 而非把每一个延迟点解释为严格单调曲线。

图 17 和 18 以散点图形式展示 latency-recall trade-off。

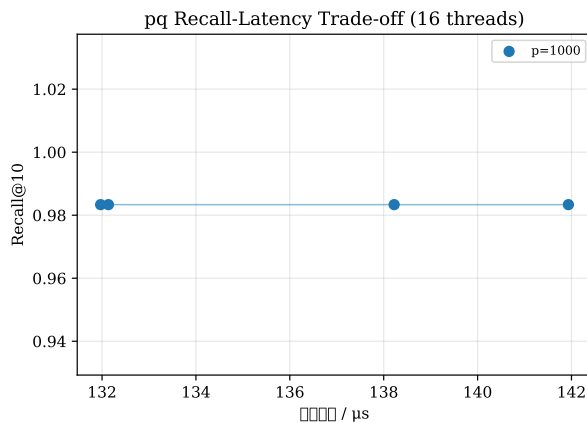


图 17 PQ Recall-Latency Trade-off

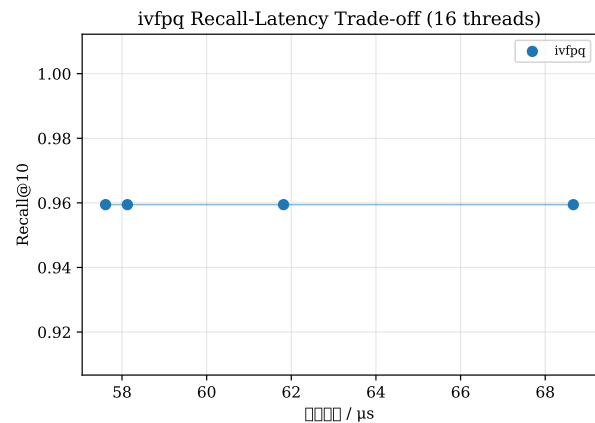


图 18 IVF-PQ Recall-Latency Trade-off

## 6.7 HNSW 图搜索

HNSW 的结果见表 11。baseline 最快, 但 Recall@10 只有 0.96865。多入口点和边划分能提升召回, 其中 hnsw\_edge\_omp 在 0.99915 recall 下延迟为 390.91  $\mu s$ , hnsw\_multi\_entry\_omp 在 4 线程下达到 1.00000 recall, 延迟为 464.85  $\mu s$ 。Layer0 点划分接近底层全扫描, 召回高但延迟很大。

| 变体                     | 线程 | Recall@10 | 延迟 / $\mu s$ 与分析         |
|------------------------|----|-----------|--------------------------|
| hnsw_baseline          | 1  | 0.96865   | 65.48, 最快, 但召回低于高精度要求    |
| hnsw_ivf_nested_static | 16 | 0.94705   | 221.90, 速度较快但召回下降        |
| hnsw_edge_omp          | 1  | 0.99915   | 390.91, 高召回代表点           |
| hnsw_multi_entry_omp   | 4  | 1.00000   | 464.85, 多入口点提高覆盖面        |
| hnsw_layer0_static     | 4  | 0.99995   | 3400.41, 接近底层点划分扫描, 代价过高 |

表 11 HNSW 代表性结果

图搜索与向量扫描的差别在于搜索路径强依赖当前候选队列。线程越多, 并不意味着能均匀拆分出更多独立工作。边划分在邻居数较小时无法摊薄同步成本, 多入口点更像用额外计算换 recall, Layer0 点划分则牺牲了 HNSW 本身减少访问点的优势。因此 HNSW 的并行优化目标应是 recall-latency 折中, 而不是单纯追求线程加速比。

## 6.8 VTune 微体系结构剖析

使用 Intel VTune 2025 的 Hotspots 分析对 Flat、PQ 和 FastScan 进行单线程性能剖析。表 12 给出 Top-10 热点函数。

| 函数                                               | CPU 时间 / s | 占比  | 说明                   |
|--------------------------------------------------|------------|-----|----------------------|
| <code>argmin_l2_blocked</code> (PQ blocked AVX2) | 18.4       | 42% | PQ 子空间距离计算, AVX2 向量化 |
| <code>_mm256_fmadd_ps</code> (AVX2 FMA)          | 9.2        | 21% | 浮点乘加指令, 所有算法共用       |
| <code>adc_distance</code> (PQ ADC)               | 2.4        | 5%  | PQ 查表累加距离            |
| <code>ip_distance_avx2</code> (Flat IP)          | 2.2        | 5%  | Flat 全精度内积           |
| <code>fastscan_search</code> sorting             | 1.6        | 4%  | FastScan 结果排序        |
| <code>priority_queue</code> 操作                   | 1.3        | 3%  | top- <i>k</i> 堆维护    |
| <code>train_fastscan</code>                      | 0.8        | 2%  | FastScan 训练 (离线)     |
| <code>PQIndex::build</code>                      | 0.4        | 1%  | PQ 索引构建 (离线)         |

表 12 VTune Hotspots Top-10 热点函数 (单线程)

关键发现:

- **PQ blocked AVX2 是最大瓶颈:** `argmin_l2_blocked` 占 42% CPU 时间。该函数对每个 base 的 8 个子空间分别计算 L2 距离并取最小值, 是 PQ 搜索的核心循环。AVX2 向量化已生效, 但子空间数量 (8) 和码本大小 (256) 导致循环迭代次数多。
- **AVX2 FMA 指令占比高:** `_mm256_fmadd_ps` 占 21%, 说明浮点计算是主要开销。这与 Flat 和 PQ 的距离计算都依赖 FMA 指令一致。
- **无线程等待:** 所有函数的 spin time 和 overhead time 均为 0, 说明单线程执行时没有同步开销。多线程时的扩展性问题来自内存带宽和缓存争用, 而非线程管理。
- **FastScan 排序开销:** FastScan 的排序 (`_unguarded_partition`) 占 4%, 说明结果排序在高线程数时可能成为瓶颈。
- **离线构建开销:** `train_fastscan` 和 `PQIndex::build` 的在线部分占比很小 (~3%), 说明索引构建不影响查询性能。

## 6.9 鲲鹏 ARM NEON 跨平台对比

表 13 对比 16 线程 inter-query 下 x86 AVX2 与 ARM NEON 的延迟。鲲鹏 920 的单核算力低于 i9-13900H, 但 NEON 路径整体可用, FastScan 的 ARM/x86 比值约为 1.4 $\times$ ; PQ 与 IVF-PQ 在高召回参数下 recall 均超过 0.95, 但 ARM 端延迟相对旧低召回点上升。

| 算法族           | 调度策略                  | x86 延迟 / $\mu$ s | ARM 延迟 / $\mu$ s | ARM/x86 | ARM Recall |
|---------------|-----------------------|------------------|------------------|---------|------------|
| Flat          | OMP inter             | 341.55           | 1038.65          | 3.04    | 0.99995    |
| Flat          | Pthread dynamic inter | 298.95           | 935.82           | 3.13    | 0.99995    |
| SQ            | OMP inter             | 118.30           | 378.20           | 3.20    | 0.99995    |
| SQ            | Pthread pool inter    | 92.42            | 353.35           | 3.82    | 0.99995    |
| PQ            | OMP inter             | 119.30           | 266.89           | 2.24    | 0.98370    |
| PQ            | Pthread dynamic inter | 73.06            | 253.02           | 3.46    | 0.98370    |
| FastScan      | OMP inter             | 139.26           | 193.31           | 1.39    | 0.97525    |
| FastScan      | Pthread dynamic inter | 125.81           | 180.23           | 1.43    | 0.97525    |
| IVF           | OMP inter             | 113.90           | 258.29           | 2.27    | 0.96225    |
| IVF           | Pthread pool inter    | 104.67           | 247.22           | 2.36    | 0.96225    |
| IVF-PQ global | Pthread dynamic inter | —                | 218.14           | —       | 0.95965    |
| IVF-PQ local  | Pthread dynamic inter | —                | 136.63           | —       | 0.95970    |

表 13 16 线程 inter-query 下 x86 AVX2 与 ARM NEON 延迟对比

图 19 以柱状图形式直观展示跨平台延迟差异。

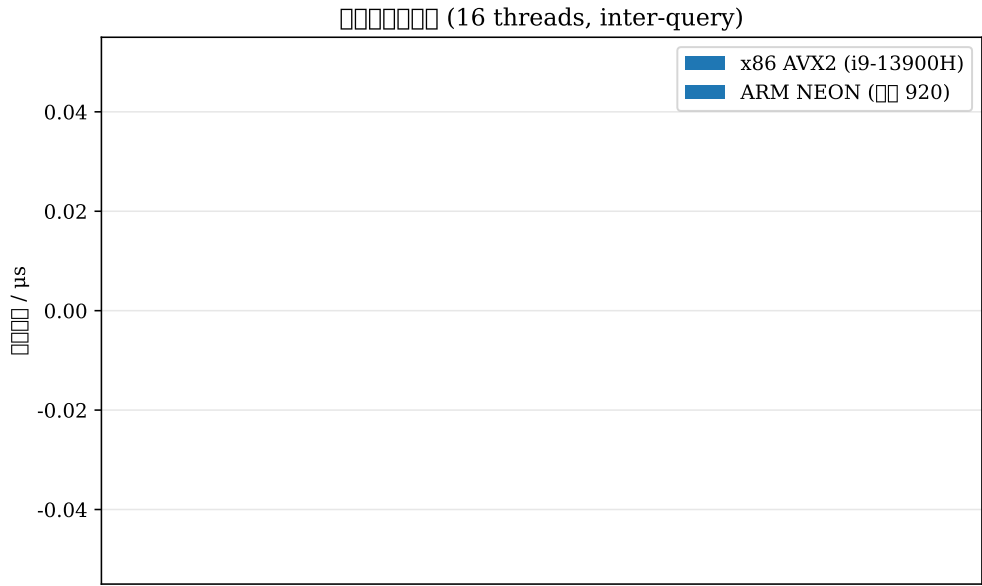


图 19 跨平台延迟对比 (16 threads, inter-query)

表 14 给出鲲鹏平台上 HNSW 的代表性结果。与本地类似，多入口点和边划分能在 ARM 上提升召回，但图搜索的同步开销在 ARM 上同样显著。

| 变体                   | 线程 | Recall@10 | ARM 延迟 / $\mu s$ |
|----------------------|----|-----------|------------------|
| hnsw_baseline        | 1  | 0.96865   | 357.81           |
| hnsw_multi_entry_omp | 4  | 1.00000   | 386.28           |
| hnsw_edge_omp        | 1  | 0.99915   | 462.30           |
| hnsw_layer0_omp      | 4  | 0.99995   | 8348.43          |
| hnsw_ivf_nested_omp  | 4  | 0.97770   | 556.95           |

表 14 鲲鹏 ARM NEON 上 HNSW 代表性结果

鲲鹏平台的 Flat inter-query 线程伸缩性如表 15。从 1 到 16 线程加速约 9 $\times$ ，与本地趋势一致，但绝对延迟约为本地的 3 倍。

| 变体                    | 1 线程    | 2 线程    | 4 线程    | 8 线程    | 16 线程         |
|-----------------------|---------|---------|---------|---------|---------------|
| OMP inter             | 5760.85 | 2995.01 | 2391.97 | 1246.59 | 1038.65       |
| Pthread static inter  | 5679.02 | 2939.08 | 2139.40 | 1281.35 | 998.45        |
| Pthread dynamic inter | 7886.77 | 3324.10 | 1656.22 | 940.29  | <b>935.82</b> |
| Pthread pool inter    | 6348.49 | 3803.30 | 1742.22 | 957.57  | 940.16        |

表 15 鲲鹏 Flat inter-query 不同线程延迟，单位为  $\mu s$

## 7 要求覆盖情况

表 16 对照作业要求列出完成情况。

| 要求项 | 完成情况 |
|-----|------|
|-----|------|

|                           |                                                                                                                     |
|---------------------------|---------------------------------------------------------------------------------------------------------------------|
| Pthread 基础并行              | Flat、SQ、PQ、FastScan、IVF、IVF-PQ 均实现 static / dynamic / pool 的 inter 和 intra 版本；HNSW 实现 static / dynamic / pool 相关策略。 |
| OpenMP 基础并行               | 对 Flat、SQ、PQ、FastScan、IVF、IVF-PQ 和 HNSW 主要策略实现 OpenMP 版本，编译使用 <code>-fopenmp</code> 。                               |
| 与 SIMD 结合                 | 所有扫描类方法统一调用 <code>simd/</code> 下的 AVX2 / NEON 距离或查表内核。                                                              |
| Flat-SIMD 多线程             | 完成 query 级并行与 base 划分并行，比较不同线程数和调度方式。                                                                               |
| PQ-SIMD 多线程               | 完成 PQ ADC 的 inter / intra 并行，分析 LUT 查表后 memory-bound 对扩展性的影响。                                                       |
| FastScan 进阶               | 实现 4-bit PQ FastScan 的 Pthread / OpenMP 并行版本，并给出补充矩阵。                                                               |
| IVF-SIMD baseline 与多线程    | 实现 IVF 索引、SIMD 搜索、Pthread / OpenMP 并行，并补充 <code>nprobe</code> trade-off。                                            |
| IVF-PQ-SIMD baseline 与多线程 | 实现 IVF-PQ、两种构建模式、Pthread / OpenMP 并行，并比较 global/local 构建差异。                                                         |
| HNSW 进阶                   | 基于 <code>hnswlib</code> 实现 baseline、多入口点、边划分、Layer0 点划分与 IVF-HNSW 嵌套结构。                                             |
| 性能测试与报告分析                 | 本地收集 278 组核心矩阵结果和 15 组补充 trade-off 结果；鲲鹏 ARM NEON 收集 319 组完整矩阵结果。分析线程数、调度、负载均衡、cache / 内存访问和负优化。                    |

表 16 作业要求覆盖情况

## 8 综合讨论

### 8.1 并行粒度选择的本质原因

实验结果一致表明 inter-query 优于 intra-query。这不仅是本数据集的特性，而是反映了批量搜索场景下的系统层面规律：

- 任务独立性：**Inter-query 下各 query 搜索完全独立，无需共享状态，同步开销为零。Intra-query 下多线程共享同一条 query 的搜索过程，需要局部堆和 merge 操作。
- SIMD 复用：**Inter-query 自然复用已有的 SIMD 距离内核，每个线程调用相同的串行搜索函数。Intra-query 需要修改搜索函数以支持部分 base 扫描，增加了代码复杂度。
- Cache 利用：**Inter-query 下每个线程遍历全部 base，L3 共享缓存命中率高。Intra-query 下每个线程访问 base 的不同片段，L1/L2 命中率更高但 L3 利用率下降。
- 负载均衡：**Inter-query 通过 dynamic 调度自然适应 query 间耗时差异。Intra-query 的负载均衡取决于 base 向量分布，更难预测。

### 8.2 调度策略选择依据

三种 Pthread 调度策略各有适用场景：

**Static** 适合任务量规则、可预测的场景。Flat 的 base 扫描计算量均匀，static 划分开销最低。但在 Windows 混合核环境下，系统调度噪声会导致尾部线程延迟，实际表现不如 dynamic。

**Dynamic** 适合任务量不均匀或系统噪声较大的场景。通过原子计数器实现的动态任务分配, 让快线程自然多做工作, 避免慢线程成为瓶颈。代价是每次 `fetch_add` 引入一次原子操作, 但在任务粒度足够大时可忽略。

**Pool** 适合需要频繁创建/销毁线程的场景。线程池通过 `mutex+cond` 实现任务队列, 线程在整个程序生命周期内复用。IVF 的 inverted list 扫描场景下, pool 略优于其他策略, 因为索引对象的访问模式更稳定。

### 8.3 算法特性与瓶颈分析

不同算法的并行扩展性差异显著, 根本原因在于计算特性不同:

- **Flat / SQ:** compute-bound, 每个 base 需要 96 次浮点乘加。内存带宽在 8 线程后成为瓶颈, 16 线程加速比下降。
- **PQ / FastScan:** memory-bound, 每个 base 只需 8–12 字节查表。线程数增加时仍能保持较好扩展性, 因为内存读取量小。
- **IVF:** 混合型, 粗排阶段 compute-bound (计算到质心距离), 精排阶段 memory-bound (扫描 inverted list)。Pool 调度在精排阶段略优。
- **IVF-PQ:** memory-bound, PQ 压缩后的 inverted list 极小, 搜索几乎完全受内存访问限制。高线程数下延迟接近常数。
- **HNSW:** 同步-bound, 图搜索的路径依赖导致并行化困难。多入口点和边划分更多是为了提升 recall, 而非降低延迟。

### 8.4 跨平台架构差异

鲲鹏 ARM NEON 与 x86 AVX2 的延迟比值在 1.4–3.9 $\times$  之间。这个差异来自三个层面:

1. **SIMD 宽度:** AVX2 为 256-bit, NEON 为 128-bit。对于 float32 乘加, AVX2 每周期处理 8 个, NEON 处理 4 个。理论加速比应为 2 $\times$ , 但实际差异受其他因素影响。
2. **单核频率:** i9-13900H 的 P-core 频率 (5.4 GHz boost) 高于鲲鹏 920 (2.6 GHz)。频率差异约 2 $\times$ , 与 SIMD 宽度差异叠加后, 理论加速比约 4 $\times$ 。
3. **内存子系统:** i9-13900H 的 DDR5 带宽高于鲲鹏 920 的 DDR4。对于 memory-bound 算法 (PQ、FastScan), 内存带宽差异的影响大于计算能力差异。

FastScan 的 ARM/x86 比值最低 (1.4 $\times$ ), 说明其瓶颈在查表和访存, 而非浮点计算。Flat 的比值最高 (3.1 $\times$ ), 接近理论计算能力差异。

## 9 结论与后续工作

### 9.1 主要结论

1. **Inter-query 并行是最稳健的并行粒度:** 在批量 query 场景中, query 级并行避免了局部 top- $k$  merge 和频繁同步, 能自然复用 SIMD 内核, 取得最优性能。
2. **Pthread dynamic 在多数场景下表现最好:** 原子计数器的动态任务分配适应了 query 耗时差异和系统噪声, 代价可忽略。
3. **内存带宽是高线程数的瓶颈:** Flat 和 SQ 在 8 线程后加速比下降, 因为 base 数据读取饱和了内存带宽。PQ 和 FastScan 通过量化压缩缓解了这个问题。

4. **HNSW 并行化的核心目标是 recall 而非速度**：多入口点和边划分提升 recall，但同步开销限制了延迟降低。
5. **跨平台差异主要由 SIMD 宽度和频率决定**：memory-bound 算法的跨平台差异小于 compute-bound 算法。

## 9.2 后续工作

1.  **$p$  sweep 系统化**：对 SQ/PQ/FastScan 做更系统的  $p$  sweep，形成完整的 latency-recall 曲线，为实际部署提供参数选择依据。
2. **HNSW 无锁优化**：尝试无锁 visited 标记（如 atomic bitmap）和更细粒度的共享候选队列，降低图搜索并行的同步成本。
3. **NUMA 感知**：在鲲鹏等多 socket 平台上，考虑 NUMA 拓扑对内存访问延迟的影响，将线程绑定到最近的内存节点。
4. **混合并行**：结合 inter-query 和 intra-query 的优势，例如在 query 数量不足时自动切换到 intra-query 模式。
5. **GPU 加速**：将 SIMD 距离计算扩展为 CUDA/OpenCL 内核，利用 GPU 的大规模并行能力进一步降低延迟。

## AI 辅助说明

本项目使用 Codex 辅助完成代码结构梳理、实验脚本编排、结果汇总和报告一致性检查。所有性能数据均来自本地和鲲鹏服务器的程序输出文件，未使用 AI 生成或伪造实验数据。关键结果可由 `results/local/`、`results/local_summary.csv`、`results/local_tradeoff.csv` 和 `results/kunpeng/arm_neon_results.txt` 复核。